VINGROUP

VINUNIVERSITY

AI in Action

Đào tạo Nhân tài AI thực chiến

Cloud Infrastructure & Deployment

Đưa Agent Lên Cloud



AICB-P1 · NGÀY 12 · VINUNIVERSITY 2026

Hãy Suy Nghĩ...

AI in Action
Đào tạo Nhân tài AI thực chiến

"Bạn demo cho sếp thấy agent chạy trên laptop.

Sếp hỏi: khi nào 100 người dùng được?

Giữ câu hỏi này trong đầu khi học bài hôm nay.

Nội Dung Bài Học

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

01 Hạ Tầng Cloud có những gì?

02 Từ LocalHost đến Production

03 Docker & Containerization

04 API gateway & security

05 Scaling & reliability

06 Cloud deployment options

07 Lab 12 + deliverable

08 Preview Day 13

Mục Tiêu Ngày 12

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

1. **Hiểu về Cloud Infrastructure:** Cloud Services, AI Architecture
2. **Hiểu gap dev → production:** dependencies, config, secrets, networking
3. **Viết Dockerfile đúng cách:** đóng gói agent thành container < 500 MB
4. **So sánh cloud options:** Railway, Render, AWS ECS, Serverless
5. **Thiết kế API gateway:** authentication, rate limiting, cost protection
6. **Deploy lên cloud:** agent có public URL hoạt động được

Deliverable Cuối Ngày

AI in Action
Đào tạo Nhân tài AI thực chiến

Agent đã được containerize và deploy lên cloud, có health check endpoint, basic authentication, và accessible qua public URL

Container

- Dockerfile (multi-stage, < 500 MB)
- docker-compose.yml + .dockerignore
- Health check endpoint /health

Deployment

- Deployed instance: Railway / Render
- Env vars đúng cách (không hardcode)
- Basic auth (API key header)

Demo

- Public URL ai cũng truy cập được
- Request → Response hoạt động
- Production readiness check ✅

01

HẠ TẦNG CLOUD CÓ NHỮNG GÌ?

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Cloud Infrastructure là gì?

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Tập hợp tài nguyên điện toán — máy chủ, mạng, lưu trữ, ảo hóa — cung cấp qua internet theo mô hình pay-as-you-go. Không cần đầu tư phần cứng, linh hoạt mở rộng theo nhu cầu thực tế.

Compute

Virtual machines, containers, serverless. Xử lý logic ứng dụng từ vài request/ngày đến hàng triệu concurrent users.

Storage

Object storage, block storage, data warehouses. Độ bền 99.999999999% — thay thế hoàn toàn on-premise storage.

Networking

VPC, Load Balancer, CDN, DNS, VPN. Kết nối toàn cầu với độ trễ tối thiểu và bảo mật nhiều lớp.

Cloud Infrastructure gồm những gì?

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Physical / Data Center
Nền tảng vật lý thực sự

Regions & AZs
Vùng địa lý độc lập, nhiều availability zones để HA

Data centers
Tòa nhà vật lý với nguồn điện, làm mát, bảo mật riêng

Bare-metal servers
Máy chủ vật lý — GPU clusters, CPU hosts

Network backbone
Cáp quang riêng nối regions, InfiniBand cho GPU clusters

Compute
Tài nguyên tính toán ảo hoá

AWS
EC2 / ECS / EKS
VMs, containers, Kubernetes

GCP
GCE / GKE / Cloud Run
VMs, K8s, serverless containers

Azure
VMs / AKS / Functions
VMs, K8s, serverless

GPU Cloud
CoreWeave / Lambda
H100/A100 clusters thuê theo giờ

Cloud Infrastructure gồm những gì?

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Storage
Lưu trữ nhiều cấp độ

Object storage
S3, GCS, Blob — dataset, checkpoint lớn, giá rẻ

Block storage
EBS, Persistent Disk — NVMe SSD tốc độ cao

File storage
EFS, Filestore — shared filesystem cho clusters

Managed DB
RDS, Cloud SQL, Aurora — PostgreSQL, MySQL

Networking
Kết nối & bảo mật mạng

VPC / Subnets
Mạng riêng ảo hoá, phân chia public/private

Load Balancer
ALB, NLB, GLB — phân tải traffic đến services

CDN
CloudFront, Cloud CDN — cache gần người dùng

DNS / Route53
Phân giải domain, routing thông minh

Security & IAM
Quyền truy cập, mã hoá, tuân thủ

3 Tầng Cloud Services

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

TẦNG 03 · CAO NHẤT

SaaS

Software as a Service

Phần mềm hoàn chỉnh, dùng ngay qua trình duyệt. Nhà cung cấp lo toàn bộ từ infrastructure đến application layer.

Gmail | Slack | Salesforce | Notion | OpenAI API | Anthropic API | Figma | Datadog

TẦNG 02 · TRUNG GIAN

PaaS

Platform as a Service

Nền tảng để deploy ứng dụng — bạn chỉ cần lo code. Runtime, OS, scaling, patching được Cloud xử lý tự động.

Cloud Run | App Engine | Heroku | Vercel | Railway | Cloud SQL | Firebase | Azure App Service

TẦNG 01 · NỀN TẢNG

IaaS

Infrastructure as a Service

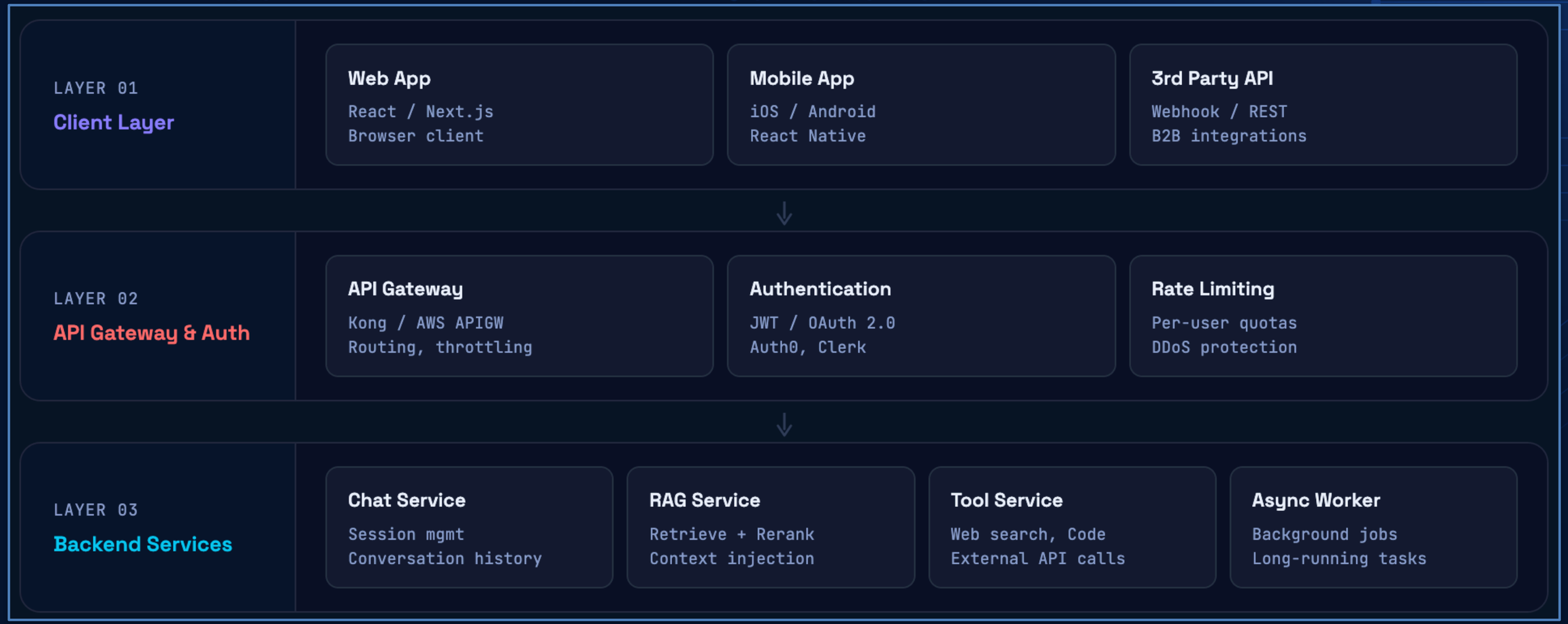
Tài nguyên thô: máy chủ ảo, mạng, ổ đĩa. Toàn quyền kiểm soát — linh hoạt nhất nhưng tốn công vận hành nhất.

EC2 | GCE | Azure VMs | S3 / GCS | VPC | ELB | EBS | CloudFront

High Level Architecture của AI Agent

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

6 Layer của 1 Ứng dụng AI hiện đại



High Level Architecture của AI Agent

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

6 Layer của 1 Ứng dụng AI hiện đại



Big 3 Cloud Providers

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

AWS, Google Cloud và Azure chiếm hơn 65% thị phần cloud toàn cầu. Mỗi nhà cung cấp có điểm mạnh riêng nhưng core services đều tương đương nhau.

AWS

COMPUTE	EC2, Lambda, ECS
STORAGE	S3, EBS, Glacier
DATABASE	RDS, DynamoDB, Aurora
AI/ML	Bedrock, SageMaker
NETWORK	VPC, CloudFront, Route53
THỊ PHẦN	~32%

Google Cloud

COMPUTE	GCE, Cloud Run, GKE
STORAGE	GCS, Persistent Disk
DATABASE	Cloud SQL, Spanner, BigQuery
AI/ML	Vertex AI, Gemini API
NETWORK	VPC, Cloud CDN, Cloud DNS
THỊ PHẦN	~12%

Microsoft Azure

COMPUTE	VMs, AKS, App Service
STORAGE	Blob, Queue, Table
DATABASE	Azure SQL, CosmosDB
AI/ML	Azure OpenAI, ML Studio
NETWORK	VNet, Azure CDN, DNS
THỊ PHẦN	~23%

2 Cloud Providers Nổi Bật

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Railway & Render là hai nền tảng nổi bật trong thế hệ PaaS mới — đơn giản hơn AWS, nhanh hơn Heroku, phù hợp cho startups, MVPs và AI apps.

Railway

Deploy in seconds, pay for what you use

Usage-based billing | Scales to zero | Per-second billing

Developer Experience

Deploy từ GitHub trong vài giây, không cần credit card. CLI mạnh mẽ, interface hiện đại. Được đánh giá là DX tốt nhất trong segment.

Pricing Model

Tính tiền theo giây dựa trên CPU + RAM thực tế dùng. App không có traffic → tốn gần như $0. Phù hợp workload không đều (dev, staging, AI batch jobs).

Infrastructure

Chạy trên Railway Metal (bare-metal tự xây) + Google Cloud. Hỗ trợ PostgreSQL, Redis, MongoDB, MySQL as managed services. GPU support: 50+ models từ RTX 3060 đến B200.

Regions

US (West/East), EU West, Asia Pacific. Chọn region per-service. Lưu ý: đã có outage ở EU West (12/2025) — uptime SLA cần kiểm tra kỹ cho production.

Render

Production-ready defaults, predictable pricing

Predictable pricing | Production-ready | Multi-region

Platform Philosophy

Unified cloud platform với production-ready defaults. Tập trung vào reliability và structured infrastructure. Documentation tốt, mature hơn Railway.

Pricing Model

Plan-based per service (không phải credit). Không có credit threshold gây dừng service bất ngờ. Render đã bỏ per-seat fees từ 2026. Dễ dự đoán ngân sách hàng tháng.

Service Types

Web Services, Background Workers, Cron Jobs, Static Sites — đều là first-class types. PostgreSQL với Point-in-Time Recovery (PITR) ở higher tiers. Redis managed.

Regions & Reliability

US (Oregon/Ohio), EU (Frankfurt), Singapore, Ohio. Multi-region tốt hơn Railway. Phù hợp apps cần EU/APAC presence. Uptime và SLA ổn định hơn cho production.

02

TỪ LOCALHOST ĐẾN PRODUCTION

Agent chạy trên máy mình ≠ Agent chạy cho 100 người

Recap: "It Works On My Machine"

AI in Action
Đào tạo Nhân tài AI thực chiến

✅ 11 Ngày Đã Build

- LLM API + prompt engineering
- RAG pipeline grounded
- Multi-agent + MCP
- UX + trust layer
- Guardrails + safety

⚠️ Nhưng Đang Chạy Trên...

- localhost:8000
- API keys trong .env file
- Chỉ 1 user (chính mình)
- Không health check
- Tắt laptop = agent chết

📌 *"It works on my machine" — câu nói nổi tiếng nhất lịch sử software engineering. Day 12 giải quyết đúng vấn đề này.*

6 Giai Đoạn từ Localhost đến Production

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Con đường từ "chạy được trên máy mình" đến "người dùng thực sự dùng được" — 6 giai đoạn, mỗi giai đoạn có bẫy riêng cần tránh.

❌ Không có quy trình

"Works on my machine" → deploy thẳng lên prod → lỗi production, downtime, data mất, khách hàng tức giận. Không có cách rollback. Bug fix khẩn cấp lúc 2 giờ sáng.

✅ Có quy trình chuẩn

Code được kiểm tra qua nhiều môi trường, CI/CD tự động hóa deploy, có staging để test, monitoring phát hiện lỗi trước người dùng. Zero-downtime deployment.

6 Giai Đoạn từ Localhost đến Production

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

BƯỚC 01

Local Development

Môi trường phát triển trên máy cá nhân. Mục tiêu: viết code nhanh, hot-reload, debug dễ. Dùng .env file cho config, Docker để simulate môi trường gần production nhất.

Tools thường dùng

Docker Compose
VS Code + DevContainer
ngrok (public tunnel)
Postman / Bruno

Best practices

Dùng .env.local
Không commit secrets
Docker từ đầu
Seed data nhất quán

Bẫy phổ biến

Hard-code localhost URL
Khác OS → khác behavior
Dùng prod DB để test
"Nó chạy được mà"

BƯỚC 02

Version Control & Code Review

Mọi thay đổi phải qua Git. Feature branch → Pull Request → Code Review → Merge. Không ai được push thẳng lên main/master. Branch protection rules là bắt buộc.

git flow | feature branches | pull request | code review | branch protection | semantic commits | GitHub / GitLab | .gitignore secrets

6 Giai Đoạn từ Localhost đến Production

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

BƯỚC 03

CI — Continuous Integration

Mỗi lần push code → pipeline tự động chạy. Nếu bất kỳ bước nào fail → không được merge. Mục tiêu: phát hiện lỗi sớm nhất có thể, trước khi vào staging hay production.

Pipeline CI điển hình

1. Lint (ESLint, Ruff)
2. Unit tests
3. Integration tests
4. Build Docker image
5. Security scan (Trivy)

Tools

GitHub Actions
GitLab CI
CircleCI
Jenkins (legacy)
Buildkite

Metrics cần đạt

Build time < 5 phút
Test coverage > 70%
0 critical CVE
Lint pass 100%

BƯỚC 04

Staging Environment

Môi trường giống production nhất có thể — cùng infrastructure, cùng config, nhưng dữ liệu là fake/ anonymized. Đây là "production thứ hai" để QA, UAT, demo khách hàng nội bộ trước khi release.

Staging phải có

Cùng Docker image
Cùng Kubernetes config
Env vars riêng biệt
Dữ liệu anonymized
SSL certificate

Tests ở staging

E2E tests (Playwright)
Load testing (k6)
Smoke tests
UAT với stakeholders
Security penetration

Triển khai staging

Railway (preview env)
Render (preview deploys)
AWS staging cluster
Vercel Preview URLs
Feature flags

6 Giai Đoạn từ Localhost đến Production

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

BƯỚC 05

CD — Deploy lên Production

Continuous Deployment / Delivery — đưa code đã được kiểm tra lên môi trường thực. Không bao giờ deploy thủ công bằng tay. Mọi thứ phải qua pipeline, có audit log, có khả năng rollback trong <5 phút.

Deploy strategies

- Rolling update
- Blue/Green deploy
- Canary release (5%→100%)
- Feature flags
- Immutable infra

Zero-downtime deploy

- Health checks
- Graceful shutdown
- DB migration first
- Backward-compat API
- readinessProbe (K8s)

Rollback plan

- git revert + redeploy
- K8s rollout undo
- DB migration down
- Feature flag off
- Hotfix branch

BƯỚC 06

Monitor & Observe Production

Deploy không phải điểm kết thúc — là điểm bắt đầu theo dõi. Production cần được quan sát liên tục: logs, metrics, traces, alerts. Mục tiêu: biết có lỗi trước khi người dùng phản ánh.

3 Pillars of Observability

- Logs → what happened
- Metrics → how much
- Traces → where slow
- (OpenTelemetry)

Tools

- Datadog / Grafana
- Sentry (errors)
- PagerDuty (alerts)
- Prometheus + Loki
- New Relic

SLAs / SLOs cần đặt

- Uptime ≥ 99.9%
- P99 latency < 500ms
- Error rate < 0.1%
- MTTR < 30 phút

So Sanh Môi Trường

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Local vs Staging vs Production

Hiểu rõ sự khác biệt giữa ba môi trường để tránh "nó chạy được ở local mà".

Đặc điểm từng môi trường

Tiêu chí	💻 Local	🧪 Staging	🚀 Production
Mục đích	Phát triển, debug	Test, QA, demo	Người dùng thực
Database	SQLite / local PG	Managed DB (fake data)	Managed DB (real data)
Secrets / Env Vars	.env.local file	.env.staging (vault)	Secret Manager / Vault
SSL / HTTPS	Thường không có	Có (Let's Encrypt)	Bắt buộc
Logging	Console.log	Structured logs	Centralized (Datadog)
Scaling	1 instance	1–2 instances	Auto-scaling
Deploy trigger	npm run dev	Merge to staging branch	Merge to main + approval
Rollback	git stash / Ctrl+Z	Redeploy previous build	K8s rollout undo <2 phút
External APIs	Mock / Sandbox keys	Sandbox keys	Live keys (rate limited)
Cost	$0	~10–20% prod cost	Full cost

6 Giai Đoạn từ Localhost đến Production

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Những lỗi phổ biến nhất

Hầu hết outage production đều xuất phát từ một trong những lỗi này.

🔑

Commit secrets lên Git

API keys, passwords, JWT secrets bị push lên GitHub. Ngay cả khi xóa, lịch sử vẫn còn. Bot scan GitHub 24/7.

→ Dùng .gitignore + Secret Manager + git-secrets hook

🐋

Không dùng Docker từ đầu

"Máy tôi Node 18, prod chạy Node 16" → bug kỳ lạ, mất nhiều giờ debug. Khác OS, khác timezone, khác locale.

→ Dockerfile ngay từ ngày đầu dự án, dùng Docker Compose local

🗄️

Dùng Production DB để dev/test

Một câu UPDATE không có WHERE → xóa hết data user. Không có môi trường test riêng là rủi ro cực kỳ cao.

→ DB riêng cho mỗi môi trường, backup hàng ngày, soft delete

📦

Không có Migration strategy

Deploy code mới nhưng quên chạy DB migration → app crash. Hoặc migration không backward-compatible → downtime.

→ Migration chạy trước deploy, luôn có migration down, test trên staging

📉

Không có Health Checks

Container "chạy" nhưng app bên trong đã crash. Load balancer tiếp tục gửi traffic vào → 100% requests fail.

→ /health endpoint, liveness + readiness probe trên K8s

🔔

Biết lỗi từ khách hàng

Không có monitoring → khách hàng báo lỗi trước team. Mất tin tưởng, mất user, MTTR cao vì không có context.

→ Sentry + Datadog + alert PagerDuty trước khi user biết

Pre Production Checklist

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Danh sách kiểm tra trước khi bấm nút deploy lên production.

🔒 Security

- ✓ **Secrets** không có trong codebase, dùng env vars hoặc Secret Manager
- ✓ **HTTPS/TLS** được bật, HTTP tự redirect sang HTTPS
- ✓ **CORS** chỉ cho phép đúng origins, không để wildcard *
- ✓ **Rate limiting** được bật ở API Gateway, tránh abuse
- ✓ **Dependency scan** không có CVE critical (Snyk, Trivy)
- ✓ **Auth** tất cả endpoints cần auth đều được bảo vệ

⚡ Performance

- ✓ **Load test** đã chạy với traffic gấp 2x dự kiến
- ✓ **DB indexes** đã thêm cho các query phổ biến, EXPLAIN ANALYZE
- ✓ **Caching** đã implement cho data ít thay đổi (Redis)
- ✓ **Assets** được compress, CDN phân phối static files
- ✓ **Connection pooling** được cấu hình đúng cho DB
- ✓ **Timeouts** đặt cho tất cả external API calls

🔄 Reliability

- ✓ **Health check endpoint** /health trả về 200 khi app sẵn sàng
- ✓ **Graceful shutdown** xử lý SIGTERM, hoàn thành requests đang chạy
- ✓ **DB migration** đã test rollback, có script migration down
- ✓ **Backup** DB backup tự động hàng ngày, đã test restore
- ✓ **Auto-scaling** được cấu hình, đã test scale-out behavior
- ✓ **Rollback plan** rõ ràng, team biết cách rollback trong <5 phút

📊 Observability

- ✓ **Centralized logging** structured logs gửi về Datadog/Grafana
- ✓ **Error tracking** Sentry được cài, alert khi error rate tăng đột biến
- ✓ **Uptime monitoring** cảnh báo khi service down (PagerDuty)
- ✓ **Dashboards** có sẵn để theo dõi CPU, memory, latency, error rate
- ✓ **Runbook** tài liệu xử lý sự cố cho các lỗi thường gặp
- ✓ **On-call rotation** ai chịu trách nhiệm khi alert lúc nửa đêm

12-Factor App — Áp Dụng Cho AI Agent

AI in Action
Đào tạo Nhân tài AI thực chiến

1 Config in env

Không hardcode API keys.
Dùng os.getenv() cho mọi config.

2 Stateless processes

Agent không giữ session
trên instance memory.

3 Port binding

Đọc PORT từ env var.
Railway/Render inject tự động.

4 Dev/prod parity

Giữ gap nhỏ nhất giữa
dev, staging, production.

Production Checklist (MVP)

- ✅ Secrets trong env vars (không trong code)
- ✅ Health check endpoint: GET /health
- ✅ Structured logging (JSON format)
- ✅ Graceful shutdown (handle SIGTERM)

Đừng cố học hết 12 factors ngay. 4 cái trên đủ cho MVP deployment.

03

DOCKER - ĐÓNG GÓI AGENT THÀNH CONTAINER

Build once — run anywhere

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Docker & Docker Compose

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Docker giải quyết bài toán cốt lõi của deployment: "chạy được trên máy tôi nhưng không chạy được trên server". Đóng gói app cùng toàn bộ dependencies vào một container duy nhất, chạy giống nhau ở mọi nơi.

🐋 Docker là gì?

Docker là nền tảng container hóa — cho phép đóng gói ứng dụng cùng toàn bộ môi trường (OS libraries, runtime, dependencies, config) thành một **image** bất biến. Image đó chạy thành **container** — nhẹ hơn VM, khởi động trong mili-giây, chạy giống nhau trên mọi máy.

Khác với VM: Docker dùng chung kernel của host OS → overhead thấp hơn nhiều. 1 máy có thể chạy hàng trăm containers song song.

📦 Dockerfile	→ Công thức / Recipe
🖼️ Image	→ Bản thiết kế đóng băng
🏃 Container	→ Instance đang chạy từ image
🗃️ Volume	→ Ổ cứng gắn ngoài container
🌐 Network	→ Mạng nội bộ giữa containers
🏪 Registry	→ Kho lưu images (Docker Hub, ECR)

Docker & Docker Compose

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Image

Snapshot bất biến của filesystem + metadata. Được build từ Dockerfile. Mỗi lệnh tạo một layer — layer được cache để build nhanh hơn.

Container

Process đang chạy được tạo từ image. Isolated về filesystem, network, process. Có thể start/stop/remove mà không ảnh hưởng image gốc.

Volume

Persistent storage nằm ngoài container lifecycle. Data trong container mất khi container bị xóa — volume giữ data tồn tại lâu dài.

Network

Mạng ảo cho phép containers giao tiếp với nhau bằng service name thay vì IP. Bridge (default), host, overlay (Swarm/K8s).

WORKFLOW

Docker Build → Ship → Run



Dockerfile

Viết recipe

→



docker build

Tạo image

→



docker run

Test local

→



docker push

Lên Registry

→



docker pull

Server kéo về

→



docker run

Production!

Docker File

AI in Action
Đào tạo Nhân tài AI thực chiến

Dockerfile

```
# ── Stage 1: Build ──
FROM node:20-alpine AS builder
```
Dùng Alpine (5MB) thay vì Ubuntu (200MB). Multi-stage: stage này chỉ để build.

```
WORKDIR /app
```
Đặt working directory, tất cả lệnh sau chạy trong /app.

```
COPY package*.json ./
```
Copy package.json TRƯỚC code — tận dụng layer cache. Nếu dependencies không đổi → không cần npm install lại.

```
RUN npm ci --only=production
```
npm ci (clean install) thay vì npm install — deterministic, nhanh hơn trong CI/CD.

```
COPY . .
```
Copy source code SAU khi install deps — code thay đổi thường xuyên, deps ít khi đổi.

```
RUN npm run build
```
Build bước này (TypeScript compile, webpack, v.v.).

```
# ── Stage 2: Production ──
FROM node:20-alpine
```
Image mới hoàn toàn — chỉ copy những gì cần thiết. Không có build tools, dev dependencies.

```
WORKDIR /app
```

```
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
```
Tạo non-root user — bảo mật quan trọng. Không bao giờ chạy app với user root.

```
COPY --from=builder /app/dist ./dist
```
Chỉ copy build output từ stage 1, không copy source hay node_modules dev.

```
COPY --from=builder /app/node_modules ./node_modules
```

```
USER appuser
```
Switch sang non-root user trước khi chạy app.

```
EXPOSE 3000
```
Khai báo port (chỉ là documentation, không tự publish port ra ngoài).

```
HEALTHCHECK --interval=30s --timeout=5s \
  CMD wget -qO- http://localhost:3000/health || exit 1
```
Health check tích hợp — Docker/K8s biết container có thực sự healthy không.

```
CMD ["node", "dist/index.js"]
```
CMD (runtime default) khác ENTRYPOINT. Dùng JSON array form — tránh shell wrapping.

Docker & Docker Compose

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

docker-compose.yml

```yaml
services:
  app:                          # AI Application
    build: .
    ports: ["3000:3000"]
    environment:
      - DATABASE_URL=postgresql://
          user:pass@db:5432/mydb
      - REDIS_URL=redis://cache:6379
      - ANTHROPIC_API_KEY=${ANTHROPIC_KEY}
    depends_on:
      db: { condition: service_healthy }
      cache: { condition: service_started }
    volumes:
      - ./src:/app/src          # hot-reload
    restart: unless-stopped

  db:                           # PostgreSQL
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: mydb
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
    volumes:
      - pg_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD","pg_isready","-U","user"]
      interval: 5s
```

🔗 DEPENDS_ON + HEALTHCHECK

App chỉ start sau khi DB báo healthy — tránh app crash vì DB chưa sẵn sàng. Dùng `service_healthy` thay vì `service_started`.

🌐 SERVICE DISCOVERY

Trong Compose network, các services gọi nhau bằng tên (`db`, `cache`). Không cần biết IP — Docker DNS tự resolve.

🔑 SECRETS QUA .ENV

`${ANTHROPIC_KEY}` được inject từ file `.env` hoặc shell environment. Không bao giờ hardcode secrets vào compose file.

💾 NAMED VOLUMES

`pg_data` và `redis_data` persist khi container restart hay rebuild. Data không mất dù `docker compose down`.

🔄 HOT RELOAD (DEV)

Mount `./src:/app/src` để code thay đổi local reflect ngay vào container — không cần rebuild image mỗi lần.

⚖️ SCALE WORKERS

`replicas: 2` chạy 2 worker instances song song xử lý queue nhanh hơn. Cũng có thể dùng `docker compose up --scale worker=4`.

```yaml
  cache:                        # Redis
    image: redis:7-alpine
    command: redis-server --maxmemory 256mb
    volumes:
      - redis_data:/data

  worker:                       # Background jobs
    build: .
    command: node dist/worker.js
    depends_on: [db, cache]
    deploy:
      replicas: 2               # 2 worker instances

volumes:
  pg_data:
  redis_data:
```

Docker Commands

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

🐋 Docker cơ bản

`docker build -t myapp:v1 .`	Build image từ Dockerfile, đặt tag myapp:v1
`docker run -p 3000:3000 myapp:v1`	Chạy container, map port 3000 host → 3000 container
`docker run -d --name api myapp:v1`	Chạy detached (background), đặt tên "api"
`docker exec -it api sh`	Vào terminal bên trong container đang chạy
`docker logs -f api`	Xem logs realtime (follow mode)
`docker ps -a`	List tất cả containers (kể cả đã dừng)
`docker images`	List tất cả images đang có
`docker system prune -a`	Dọn dẹp images/containers không dùng, giải phóng disk

🧩 Docker Compose

`docker compose up -d`	Khởi động toàn bộ stack, chạy background
`docker compose up --build`	Rebuild images rồi mới start (sau khi sửa code)
`docker compose down`	Dừng và xóa containers (volumes vẫn giữ)
`docker compose down -v`	Dừng + xóa cả volumes (reset hoàn toàn)
`docker compose logs -f app`	Xem logs của service "app" realtime
`docker compose exec db psql -U user`	Vào psql CLI của PostgreSQL service
`docker compose scale worker=4`	Tăng lên 4 instance của worker service
`docker compose ps`	Xem trạng thái tất cả services trong stack

Docker & Docker Compose

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Tối ưu Docker cho AI Apps

✓ MULTI-STAGE BUILD

```
FROM node:20-alpine AS builder
RUN npm ci && npm run build

FROM node:20-alpine
COPY --from=builder /app/dist .
# Image size: ~120MB
```

Tách build stage và runtime stage — image production nhỏ hơn 5-10x, không có dev tools hay source code.

✗ COPY TRƯỚC, INSTALL SAU

```
COPY . .              # SAI
RUN npm install       # Cache miss mọi lần

# Đúng: copy package.json TRƯỚC
COPY package*.json ./
RUN npm ci
COPY . .
```

Tận dụng layer caching: dependencies thay đổi ít hơn code → đặt COPY package.json trước để cache tầng npm install.

✓ .DOCKERIGNORE

```
node_modules/
.git/
.env*
*.log
dist/
coverage/
README.md
.DS_Store
```

Tương tự .gitignore — loại trừ files không cần

✓ NON-ROOT USER

```
RUN addgroup -S app   && adduser -S app -G app

USER app

# KHÔNG bao giờ chạy:
USER root
CMD ["node", "index.js"]
```

Container chạy root = nếu bị exploit, attacker có root access vào host. Non-root user là security baseline tối thiểu.

✓ PIN IMAGE VERSIONS

```
FROM node:20.15.1-alpine3.20
FROM postgres:16.3-alpine

# Tránh dùng:
FROM node:latest
FROM node:lts
```

Tag "latest" thay đổi bất cứ lúc nào → build hôm nay khác build tuần sau. Pin exact version để reproducible builds.

✓ HEALTH CHECK

```
HEALTHCHECK   --interval=30s   --t
```

Docker và Kubernetes dùng health check để

04

API GATEWAY & SECURITY

Bảo vệ agent trước khi request đến được logic

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

API GATEWAY & SECRITY

AI in Action
Đào tạo Nhân tài AI thực chiến

API Gateway là "cổng vào duy nhất" của mọi hệ thống hiện đại — xử lý auth, rate limiting, routing, logging trước khi request đến bất kỳ service nào. Kết hợp với lớp bảo mật đa tầng để bảo vệ ứng dụng AI.

🔀 API Gateway là gì?

Reverse proxy thông minh đứng trước tất cả backend services. Thay vì client gọi trực tiếp vào từng service, mọi request đều đi qua một điểm duy nhất — giúp centralize cross-cutting concerns: auth, logging, rate limiting, SSL termination, request transformation.

🛡️ Tại sao cần Security Layer?

AI apps đặc biệt nhạy cảm: prompt injection, model abuse (chi phí cao), data exfiltration qua LLM, PII leakage trong responses. Một request độc hại không bị chặn có thể tiêu tốn hàng trăm USD API costs hoặc lộ dữ liệu người dùng. Security phải là tầng đầu tiên, không phải afterthought.

🔄 Vòng đời một request qua API Gateway



API GATEWAY CORE FEATURE

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

⏱️

Rate Limiting

Giới hạn số request theo user, IP, API key, tier. Bảo vệ backend khỏi bị quá tải và tránh abuse LLM API (tiết kiệm chi phí).

```
100 req/min → free tier 1000 req/min → pro tier 10 req/min → unauthenticated
```

🔀

Routing & Load Balancing

Điều hướng request đến đúng service dựa trên path, method, header. Round-robin hoặc weighted routing giữa các instances.

```
/api/v1/chat → chat-svc /api/v1/search → rag-svc /api/v2/* → new-backend
```

🪪

Authentication

Verify JWT token, API key, OAuth token trước khi request đến service. Service không cần tự implement auth logic nữa.

```
Authorization: Bearer <jwt> X-API-Key: sk-live-xxxxx → Gateway verify, inject user_id
```

🔄

Request/Response Transform

Thêm, xóa, đổi headers. Convert XML↔JSON. Inject metadata như user_id, tenant_id vào request trước khi forward.

```
Add: X-User-Id: 12345 Add: X-Tenant: acme-corp Remove: Authorization header
```

📋

Logging & Tracing

Log mọi request tập trung: latency, status code, user, path. Distributed tracing với trace_id để debug across services.

```
trace_id: abc-123 latency: 234ms status: 200 | user: u_456
```

⚡

Caching

Cache response của các request giống nhau (GET). Đặc biệt hữu ích cho AI apps: cache embedding lookups, FAQ responses.

```
Cache-Control: max-age=300 ETag: "abc123" → 304 Not Modified (instant)
```

AUTHENTICATION

AI in Action
Đào tạo Nhân tài AI thực chiến

JWT vs API Key — Khi nào dùng gì?

🪪 JWT Flow (User Authentication)

1. **User login**
   POST /auth/login với email + password. Server verify credentials.
2. **Server trả JWT**
   Tạo access_token (15 phút) + refresh_token (7 ngày). Ký bằng secret key.
3. **Client gửi JWT**
   `Authorization: Bearer <access_token>` trong mọi request.
4. **Gateway verify**
   Decode JWT, verify signature, check expiry, extract user_id + roles. Không cần gọi DB.
5. **Token hết hạn → Refresh**
   Dùng refresh_token để lấy access_token mới, không cần login lại.

🔑 API Key Flow (Service / B2B)

1. **Developer tạo API Key**
   Qua dashboard hoặc API. Server tạo `sk-live-xxxxxxxx`, hash rồi lưu DB.
2. **Client gửi API Key**
   `X-API-Key: sk-live-xxx` hoặc trong Authorization header.
3. **Gateway lookup**
   Hash key, lookup trong Redis cache (fast path) hoặc DB. Verify key còn active.
4. **Rate limit per key**
   Mỗi API key có quota riêng theo plan. Theo dõi usage, billing.
5. **Rotate & Revoke**
   Key bị compromise → revoke ngay lập tức. Hỗ trợ multiple keys per account để zero-downtime rotation.

AUTHENTICATION

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Giải phẫu JWT Token

JWT = Header.Payload.Signature (Base64URL encoded)

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ1c2VyXzEyMyIsIm5hbWUiOiJOZ3V5ZW4gVmFuIEEiLCJyb2xlIjoicHJvIiwiaWF0IjoxNzE3MTIwMDAwLCJleHAiOjE3MTcxMjA5MDB9.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

HEADER

```
{ "alg": "HS256", "typ": "JWT" }
```

PAYLOAD (Claims)

```
{ "sub": "user_123", "name": "Nguyen Van A", "role": "pro", "iat": 1717120000, "exp": 1717120900 }
```

SIGNATURE

```
HMACSHA256( base64(header) + "." + base64(payload), SECRET_KEY ) // Không thể giả mạo // nếu không có key
```

Chiến lược Rate Limiting cho AI Apps

📊 Quota theo Plan (ví dụ AI chat app)

Giới hạn theo tier người dùng — bảo vệ LLM cost đồng thời tạo incentive upgrade.

Free	10/ngày
Starter	100/ngày
Pro	500/ngày
Enterprise	Unlimited

⚙️ Các thuật toán Rate Limiting

Mỗi thuật toán có trade-off riêng về độ chính xác, memory, và burst handling.

Token Bucket

Cho phép burst ngắn. Bucket có capacity N, refill theo thời gian. Phổ biến nhất. Dùng cho: API request quotas.

Sliding Window

Chính xác hơn Fixed Window. Track timestamp của từng request trong window. Tốt cho: giới hạn chặt chẽ.

Leaky Bucket

Output rate cố định bất kể input rate. Smooth traffic, no burst. Tốt cho: downstream service protection.

API GATEWAY CHỌN TOOL NAO

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Kong

OPEN SOURCE · SELF-HOSTED

- ✓ Plugin ecosystem phong phú
- ✓ Performance cực cao (Nginx core)
- ✓ Kubernetes Ingress controller
- · Cần tự vận hành
- · Config phức tạp

AWS API Gateway

MANAGED · AWS ECOSYSTEM

- ✓ Zero ops, fully managed
- ✓ Native Lambda integration
- ✓ Auto-scaling built-in
- · Vendor lock-in AWS
- · Cost cao ở high traffic

Cloudflare

EDGE · GLOBAL CDN + WAF

- ✓ DDoS protection tốt nhất
- ✓ Edge network 300+ PoPs
- ✓ Workers cho edge logic
- · Ít linh hoạt hơn Kong
- · Cost leo thang khi scale

Traefik

OPEN SOURCE · CLOUD-NATIVE

- ✓ Auto-discover K8s services
- ✓ Let's Encrypt tự động
- ✓ Dashboard đẹp, dễ debug
- · Plugin ít hơn Kong
- · Tài liệu phức tạp

05

SCALING & RELIABILITY

Agent không chết khi có 100 người dùng cùng lúc

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

SCALING & RELIABILITY

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Vertical vs Horizontal Scaling

⬆ Vertical Scaling (Scale Up)

Nâng cấp phần cứng của một máy — thêm CPU, RAM, SSD nhanh hơn



1 server lớn thay vì nhiều server nhỏ

Cách làm	t3.micro → t3.2xlarge
Downtime	Thường cần restart
Giới hạn	Có ceiling (max CPU/RAM)
Cost	Non-linear, đắt dần
Phù hợp	DB, legacy apps, quick fix
Không phù hợp	Long-term, HA required

↔ Horizontal Scaling (Scale Out)

Thêm nhiều máy chạy song song — phân tải đều qua load balancer



Nhiều server nhỏ → dễ thêm bớt

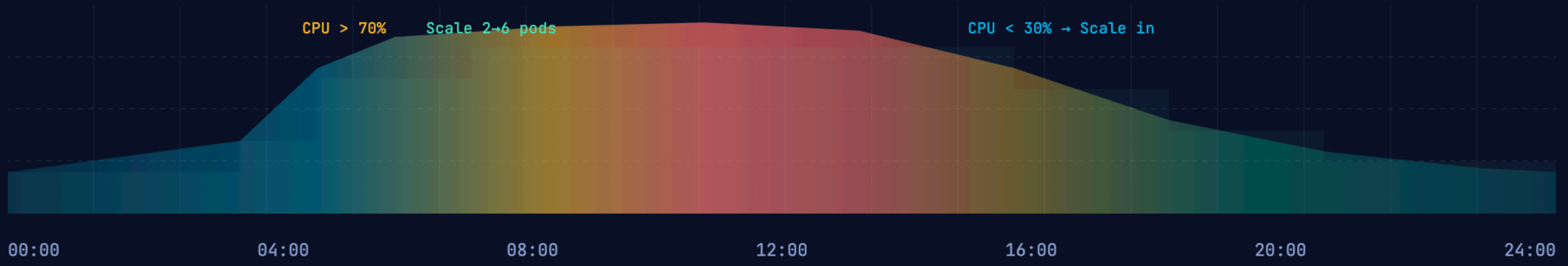
Cách làm	Thêm instance / pod
Downtime	Zero downtime
Giới hạn	Gần như vô hạn
Cost	Linear, dễ dự đoán
Phù hợp	Stateless services, AI inference
Yêu cầu	App phải stateless

SCALING & RELIABILITY

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Kubernetes HPA — Auto-scaling theo traffic

📈 Traffic spike → K8s tự scale out → spike qua → scale in



MIN PODS	MAX PODS	SCALE TRIGGER	SCALE TIME
2	20	70% CPU utilization	~90s new pod ready

SCALING & RELIABILITY

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Chiến lược Zero-Downtime Deployment

Rolling Update

Thay thế instances cũ từng cái một. Luôn có instances đang phục vụ. Default strategy của Kubernetes.



Downtime	Zero
Rollback	Chậm hơn
Resources	Bình thường
Risk	v1+v2 cùng lúc

Blue / Green

Duy trì hai môi trường giống hệt nhau. Switch traffic bằng một thay đổi DNS / load balancer. Rollback tức thì.



Downtime	Zero
Rollback	Tức thì <1 phút
Resources	Gấp đôi (tạm thời)
Risk	Thấp nhất

Canary Release

Đưa version mới cho một phần nhỏ users (5–10%) trước. Theo dõi metrics, nếu ổn mới rollout 100%.



Downtime	Zero
Risk control	Tốt nhất
Phức tạp	Cao nhất
Observability	Cần tốt

Thuật toán Load Balancing

Round Robin



Phân phối đều tuần tự. Đơn giản nhất. Giả định mọi request tốn thời gian như nhau.

→ Stateless APIs đồng đều

Least Connections



Gửi request đến server có ít connections nhất. Tốt hơn Round Robin khi request duration khác nhau.

→ Long-running requests, AI inference

IP Hash



Cùng IP → cùng server (session stickiness). Đảm bảo user luôn vào cùng một instance.

→ Stateful apps, WebSocket

Weighted



Server mạnh hơn nhận nhiều traffic hơn theo tỷ lệ. Canary deployment dùng cách này (90/10).

→ Mixed hardware, canary release

SCALING & RELIABILITY

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Design Patterns cho Hệ thống Tin cậy

Circuit Breaker

Khi downstream service lỗi liên tục → "ngắt mạch" tạm thời, không tiếp tục gọi vào service đó. Sau một khoảng thời gian thử lại (half-open). Ngăn cascade failure lan rộng.

```
CLOSED  → request bình thường
  ↓ error rate > 50%
OPEN    → reject tất cả request
  ↓ sau 30 giây
HALF-OPEN → thử 1 request
  ↓ success → CLOSED lại
```

Retry với Exponential Backoff

Thử lại request khi thất bại, nhưng tăng dần thời gian chờ giữa các lần (+ jitter ngẫu nhiên để tránh thundering herd). Không bao giờ retry vô hạn.

```
attempt 1: fail → wait 1s
attempt 2: fail → wait 2s
attempt 3: fail → wait 4s
attempt 4: success ✓
max_retries = 3, jitter = ±20%
```

Bulkhead

Cô lập tài nguyên theo loại request — như vách ngăn tàu thủy. Thread pool riêng cho mỗi service. Một service chậm không làm chết thread pool chung.

```
AI_POOL:      max 20 threads
DB_POOL:      max 10 threads
EXTERNAL_POOL: max 5 threads
→ AI slow ≠ DB slow
```

Timeout

Không chờ vô hạn. Mọi external call phải có timeout. LLM call có thể mất 30s+ → cần stream response và timeout hợp lý cho từng loại.

```
connect_timeout:  2s
read_timeout:     30s  # LLM stream
db_timeout:       5s
external_api:     10s
```

Idempotency

Request giống nhau gọi nhiều lần → kết quả giống nhau. Quan trọng khi retry: không tạo duplicate records. Dùng idempotency key trong header.

```
POST /payments
Idempotency-Key: uuid-123
→ gọi lần 1: tạo payment
→ gọi lần 2: trả lại kết quả cũ
```

Health Check & Self-healing

Kubernetes liên tục probe liveness + readiness. Container unhealthy → tự restart. Pod không ready → lấy ra khỏi load balancer. Không cần can thiệp thủ công.

```
livenessProbe:  /health → restart if fail
readinessProbe: /ready  → remove from LB
startupProbe:   /startup → slow init ok
```

SCALING & RELIABILITY

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Đo lường độ tin cậy

SLI

SERVICE LEVEL INDICATOR

Chỉ số đo lường thực tế — số liệu quan sát được từ hệ thống đang chạy. Là raw data.

```
availability = successful_req / total_req * 100 p99_latency = 234ms error_rate = 0.08%
```

SLO

SERVICE LEVEL OBJECTIVE

Mục tiêu nội bộ team tự đặt ra. Thường cao hơn SLA một chút — "error budget" để có buffer trước khi vi phạm SLA.

```
Uptime target: 99.9% P99 latency: < 500ms Error rate: < 0.1% Error budget: 8.7h/year
```

SLA

SERVICE LEVEL AGREEMENT

Cam kết pháp lý với khách hàng. Vi phạm SLA → hoàn tiền, penalty. Thường thấp hơn SLO thực tế.

```
Guaranteed: 99.5% Violation: credit back Response time: <4h critical Compensation: 10-30% bill
```

Kiểm tra hệ thống bằng cách... cố tình phá nó

Netflix nổi tiếng với **Chaos Monkey** — tool tự động kill random server trong production. Triết lý: thà tự phá có kiểm soát để tìm điểm yếu, còn hơn để production thật phá bất ngờ.

💥 Các loại Chaos Experiments

Mô phỏng failure scenarios có thể xảy ra trong thực tế

- 🔌 **Pod kill:** Xóa random pod → K8s có tự restart và traffic không bị ảnh hưởng?
- 🦕 **Network latency:** Inject 2s delay vào calls đến DB → app có timeout/fallback đúng?
- 💾 **Memory pressure:** Fill RAM đến 90% → có OOM killer? App có degrade gracefully?
- 🌐 **Dependency down:** Tắt Redis/DB → circuit breaker kích hoạt, fallback cache chạy?
- 📦 **Disk full:** Fill /tmp → logging crash? App vẫn phục vụ được không?

🛡️ Kết quả kỳ vọng & Tools

Hệ thống resilient phải pass được các bài test này

- ✅ **Auto-recovery:** K8s restart pod trong <90s, traffic tự redirect qua healthy pods.
- ✅ **Circuit breaker:** Sau 5 lỗi liên tiếp → open circuit, trả fallback response ngay.
- ✅ **Graceful degradation:** Cache miss → serve stale data thay vì error 500.
- 🔧 **Tools:** Chaos Monkey, LitmusChaos (K8s native), Gremlin (enterprise), AWS FIS.
- 📋 **GameDay:** Chạy chaos experiments có kế hoạch, team sẵn sàng, rollback plan chuẩn bị trước.

04

CLOUD DEPLOYMENT OPTIONS

Chọn platform dựa trên traffic, cost, control, compliance

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

3 Tier Deployment

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Tier 1

Railway / Render / Fly.io

- < 10 phút deploy
- Free tier có sẵn
- Zero config
- MVP / Demo / Học

BẮT ĐẦU ĐÂY

Tier 2

AWS ECS / GCP Cloud Run

- Enterprise-grade
- Auto-scaling
- CI/CD pipeline
- Production ready

KHI CẦN SCALE

Tier 3

Kubernetes (Self-managed)

- Full control
- Complex ops
- Large-scale
- Multi-cloud

KHI LỚN HƠN

So Sánh Platform

AI in Action
Đào tạo Nhân tài AI thực chiến

	Railway	Render	Cloud Run	Lambda
Deploy time	< 5 phút ✅	< 10 phút	10-15 phút	15-20 phút
Pricing	Usage-based	Free tier	Per-request	Per-invoke
Scaling	Auto ✅	Auto ✅	Auto ✅	Auto ✅
Complexity	🟢 Thấp	🟢 Thấp	🟡 Trung bình	🟡 Trung bình
Best for	MVP, demo	Side project	Production	Low-traffic

⚠️ *Serverless cold start 5-15s. Với AI agent cần response nhanh, container-based (Railway/Render) thường tốt hơn Lambda.*

Railway — Deploy Trong 5 Phút

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

1 **Kết nối GitHub repo**

```
railway login && railway init
```

2 **Railway tự detect Dockerfile / Nixpacks**

3 **Set environment variables**

```
railway variables set OPENAI_API_KEY=sk-...
```

4 **Click Deploy (hoặc railway up)**

```
railway up
```

5 **Nhận public URL** 🎉

```
railway domain  →  your-app.up.railway.app
```

railway.toml

```
[build]
builder = "NIXPACKS"

[deploy]
startCommand = "uvicorn app:app
  --host 0.0.0.0 --port $PORT"
healthcheckPath = "/health"
restartPolicyType = "ON_FAILURE"
```

Đặt PORT=8000 trong env vars. Railway inject PORT tự động — agent PHẢI đọc os.getenv('PORT').

CICD

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

CI/CD tự động hoá toàn bộ hành trình từ khi developer push code đến khi code chạy trên production — không cần thao tác thủ công, không sợ quên bước nào, deploy nhanh và an toàn hơn nhiều lần.

🔄 CI — Continuous Integration

Mỗi lần developer push code lên Git → pipeline tự động chạy ngay: build, lint, test, security scan. Nếu bất kỳ bước nào fail → merge bị chặn. Mục tiêu: phát hiện lỗi **càng sớm càng tốt**, khi còn rẻ để sửa.

Rule vàng: CI phải chạy xong trong **< 10 phút**. Nếu lâu hơn, developer mất kiên nhẫn và bỏ qua.

🚀 CD — Continuous Delivery / Deployment

Delivery: Artifact đã tested, sẵn sàng deploy bất cứ lúc nào — nhưng cần người bấm nút approve. Phù hợp với team cần kiểm soát thủ công trước khi release.

Deployment: Mọi thứ hoàn toàn tự động — merge vào main là deploy lên production ngay, không cần ai approve. Phù hợp với team có test coverage cao và monitoring tốt.

Pipeline đầy đủ — từ git push đến production

⚡ Triggered on: push to main / pull request

📩 Trigger

- ✓ git push / PR
- ✓ Checkout code
- ✓ Setup env
- ✓ Cache restore

⏱ ~15s

🔍 Lint & Format

- ✓ ESLint / Ruff
- ✓ Prettier check
- ✓ Type check (tsc)
- ! Import sort

⏱ ~30s

🧪 Test

- ✓ Unit tests
- ✓ Integration tests
- ✓ Coverage >70%
- ! Snapshot tests

⏱ ~2m

🔒 Security

- ✓ Trivy (CVE scan)
- ✓ Snyk deps audit
- ✓ Secret scan
- ✗ Block if critical

⏱ ~45s

🐋 Build & Push

- ✓ docker build
- ✓ Tag with SHA
- ✓ Push to ECR/GCR
- ✓ Scan image

⏱ ~2m

🧩 Deploy Staging

- ✓ kubectl apply
- ✓ Run migrations
- ✓ Smoke tests
- ✓ E2E (Playwright)

⏱ ~3m

✅ Approve

- ! Manual review
- ✓ QA sign-off
- ✓ Changelog ready
- ! Notify team

⏱ manual

🚀 Deploy Prod

- ✓ Canary 5%
- ✓ Monitor 10m
- ✓ Rollout 100%
- ✓ Alert if error↑

⏱ ~5m

Total (auto): ~8–10 phút

06

GITHUB PROJECT WALKTHROUGH

Ví dụ cơ bản + chuyên sâu cho từng section

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Project Structure — day12-agent-deployment/

AI in Action
Đào tạo Nhân tài AI thực chiến

01 **01-localhost-vs-production/**

basic/: Anti-patterns · advanced/: 12-Factor compliant

02 **02-docker/**

basic/: Single-stage · advanced/: Multi-stage + Compose + Nginx

03 **03-cloud-deployment/**

railway/ · render/ · advanced-cloud-run/ (CI/CD)

04 **04-api-gateway/**

basic/: API Key · advanced/: JWT + Rate Limiter + Cost Guard

05 **05-scaling-reliability/**

basic/: Health checks · advanced/: Stateless + Redis + 3 replicas

06 **06-lab-complete/**

Production-ready agent · check_production_ready.py

Key Files — Localhost vs Production

AI in Action
Đào tạo Nhân tài AI thực chiến

Section 4: API Gateway

`basic/app.py` `→ API Key auth (30 lines)`	→ nâng cấp	`advanced/auth.py + rate_limiter.py + cost_guard.py` `→ JWT + Sliding Window + Budget cap`

Section 5: Scaling

`basic/app.py` `→ /health + /ready + graceful shutdown`	→ nâng cấp	`advanced/app.py + docker-compose.yml` `→ Redis sessions + 3 replicas + Nginx LB`

utils/mock_llm.py → Dùng chung cho tất cả sections (không cần API key thật)

LAB

LAB 12
CONTAINERIZE & DEPLOY

Mục tiêu: agent có public URL ai cũng truy cập được

AI IN ACTION
Đào tạo Nhân tài AI thực chiến

Lab 12 — Các Bước Thực Hiện

AI in Action
Đào tạo Nhân tài AI thực chiến

1 **Viết Dockerfile**

Multi-stage build, non-root user, HEALTHCHECK. Target < 500 MB.

2 **Build & Test Local**

docker build -t my-agent . → docker run -p 8000:8000 my-agent

3 **Deploy Railway / Render**

Connect GitHub → Set env vars → Deploy → Nhận public URL

4 **Health Check Endpoint**

GET /health trả về: status, uptime, version, dependencies

5 **Basic Authentication**

API Key header: X-API-Key. Reject 401 nếu thiếu hoặc sai key.

6 **Demo Request → Response**

curl -H 'X-API-Key: ...' -X POST https://your-url/ask -d '{"question":"..."}'

Blueprint Cần Nộp

AI in Action
Đào tạo Nhân tài AI thực chiến

Không cần enterprise-grade. Cần chứng minh: biết đưa agent từ localhost lên cloud và nó hoạt động.

📦 Container

✅ Dockerfile
Multi-stage build, < 500 MB, non-root user

✅ docker-compose.yml
Agent + dependencies

✅ .dockerignore
Loại bỏ .env, __pycache__, venv/

✅ GET /health
Trả về status + uptime + version

✅ GET /ready
Readiness check

☁️ Deployment

✅ Public URL
https://your-app.up.railway.app

✅ Env vars
Secrets trong env, không hardcode

✅ API Key auth
X-API-Key header, 401 nếu thiếu

✅ Demo
Request → Response qua public URL

✅ check_production_ready.py
20/20 checks pass ✅

Key Takeaways

AI in Action
Đào tạo Nhân tài AI thực chiến

1 **Container = "ship anywhere"**

Docker giải quyết "it works on my machine". Multi-stage + slim image → agent < 500 MB.

2 **Start simple, migrate later**

Railway/Render cho MVP (< 10 phút). Migrate AWS/GCP khi business thật sự cần scale.

3 **Security from day 1**

Secrets management + HTTPS + rate limiting + spending caps. Setup trước khi có user thật.

4 **Health check + rolling deploy = always available**

Stateless design + Redis cho phép scale bằng cách thêm instances bất kỳ lúc nào.

Preview — Day 13: Monitoring & Observability

AI in Action
Đào tạo Nhân tài AI thực chiến

"Agent deploy xong, 3 ngày sau: latency tăng gấp đôi, cost tăng 300%.
Bạn không biết cho đến khi user phàn nàn."

📊 Metrics

Latency, throughput, error rate, token usage

📝 Logging

Structured logs → Loki / Datadog / CloudWatch

🔍 Tracing

LangSmith / Langfuse cho LLM traces

🚨 Alerting

PagerDuty / Slack alert khi anomaly detected

Chuẩn bị: Đọc LangSmith hoặc Langfuse quickstart (20 phút) trước buổi sau.

VINGROUP

VINUNIVERSITY

Từ hôm nay, agent không còn chỉ chạy trên máy bạn.

Nó đã là một service thật sự.

AICB-P1 · Ngày 12 · VinUniversity 2026

